Aprendizaje Supervisado

Índice

Introducción 01

Regresion Lineal 02

Regresión polinómica 03

Regresión Logística 04

Árboles de decisión 05

SVM 06

Naive Bayes 07

KNN 08

Ensemble models 09

- Random Forest
- GBM

Aprendizaje supervisado

Aprendizaje supervisado son un conjunto de técnicas de Machine Learning (ML) donde un modelo se entrena usando un conjunto de datos de entrada (features) junto con sus correspondientes salidas deseadas (labels). Este conjunto de datos etiquetado actúa como una "guía", enseñando al modelo a reconocer patrones y asociar las características de entrada con las salidas correctas.

El proceso implica dos etapas principales:

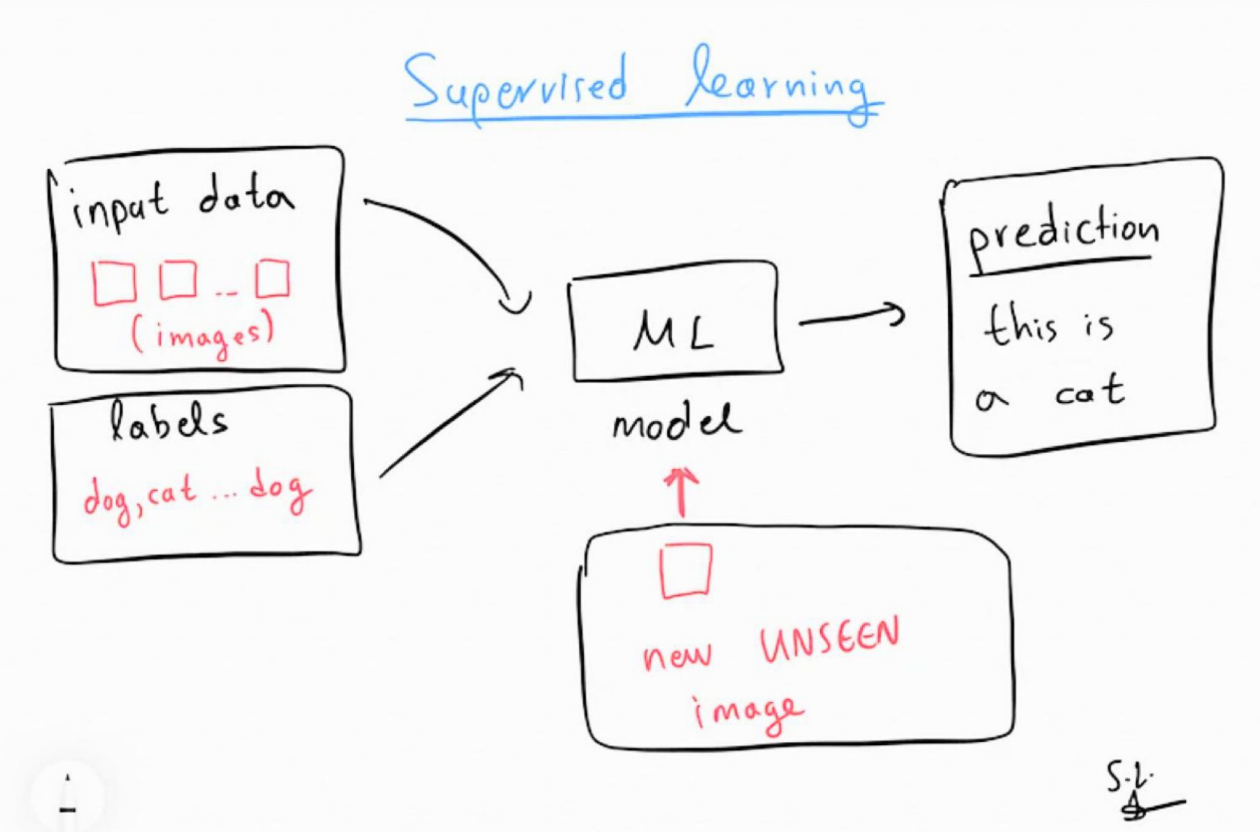
1. **Entrenamiento:** El modelo analiza las relaciones entre las entradas y las salidas etiquetadas en los datos de entrenamiento. Por ejemplo, en un modelo de clasificación, aprenderá a categorizar datos en clases específicas basándose en estas asociaciones.

2. **Predicción/Evaluación:** Una vez entrenado, el modelo se pone a prueba con datos nuevos o no vistos (sin etiquetas visibles para el modelo) para predecir la salida correcta.

Ejemplos comunes de aprendizaje supervisado incluyen:

- **Clasificación:** Predecir categorías, por ejemplo, detectar correos electrónicos como "spam" o "no spam".
- **Regresión:** Predecir valores continuos, como el precio de una casa en función de sus características.

Aprendizaje supervisado



Regresión Lineal

En la regresión lineal de lo que se trata es de predecir valores continuos: por ejemplo en banca se utiliza para estimar la renta de un cliente o para determinar la elasticidad al precio de la demanda de hipotecas.

La regresión lineal trata de predecir el valor de la variable objetivo a partir de estimar unos pesos para cada una de las variables input más un término independiente (intercept)

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_n x_n$$

$$x_i = variables\ input;\ \ \theta_i = coeficientes$$

La clave de la regresión lineal está en encontrar los valores óptimos de los coeficientes que hacen mínimo el valor del error cuadrático medio (MSE):

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

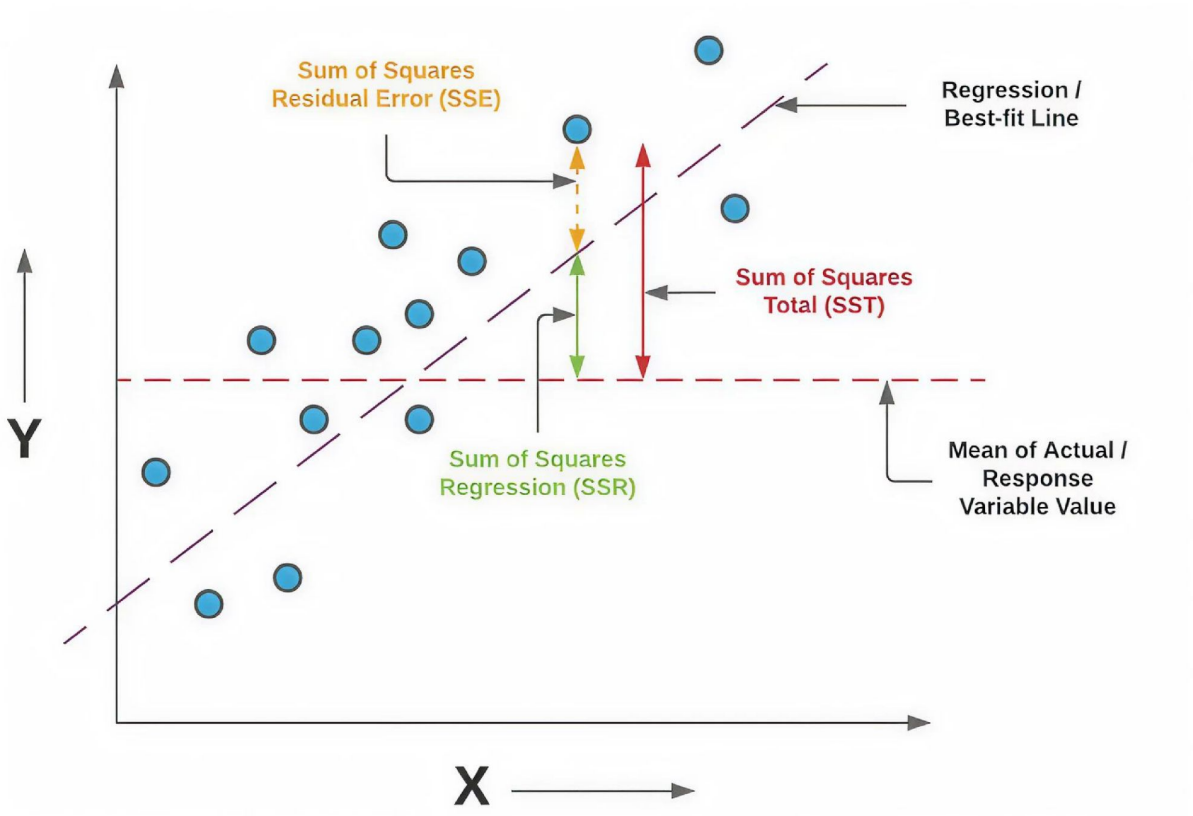
Regresión Lineal

La ecuación normal determina los coeficientes que hacen mínima la función de coste (MSE)

$$\hat{\theta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

```
lm = LinearRegression()
lm.fit(X_train,y_train)
```

Regresión Lineal: Métricas



$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}}$$

$$\text{SSE} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

$$\text{SST} = \sum_{i=1}^{n} (y_i - \bar{y})^2$$

$$SSR = \sum_{i=1}^{n} (\hat{y}_i - \bar{y})^2$$

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Regresión Lineal: Ridge y Lasso

Ridge Regression (Regularización L2): Agrega un término de penalización basado en la norma L2 de los coeficientes del modelo (el cuadrado de la magnitud de los coeficientes).

Penaliza los coeficientes grandes y tiende a reducir la magnitud de todos los coeficientes hacia 0.

$$MSE = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^{n} \theta_j^2$$

Lasso Regression (Regularización L1): agrega un término de penalización basado en la norma L1 de los coeficientes (la suma de los valores absolutos de los coeficientes).

Puede reducir algunos coeficientes a exactamente 0. Esto lleva a selección de características automática, es decir, el modelo descarta las variables menos relevantes.

$$MSE = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^{n} |\theta_j|$$

Regresión Lineal: Ridge y Lasso

```
from sklearn.linear_model import Ridge, Lasso
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Ridge
ridge = Ridge(alpha=1.0)  # 'alpha' es el parámetro de regularización equivalente a 'lambda'
ridge.fit(X_train, y_train)
y_pred_ridge = ridge.predict(X_test)
print("Ridge MSE:", mean_squared_error(y_test, y_pred_ridge))

# Lasso
lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)
y_pred_lasso = lasso.predict(X_test)
print("Lasso MSE:", mean_squared_error(y_test, y_pred_lasso))
```

Regresión lineal: Estimación de parámetros

Cuando el número de variables input es muy grande el coste de computar la ecuación normal se vuelve muy alto. Entonces aplicamos el método del descenso del gradiente para intentar llegar al óptimo con SGDRegressor. Además también permite incluir regularización (l1, l2, elasticnet)

```
from sklearn.linear_model import SGDRegressor

model = SGDRegressor(max_iter=1000, learning_rate='optimal', penalty='l2')
model.fit(X_train, y_train)
```

En resumen:

- Usa SGDRegressor si trabajas con grandes volúmenes de datos o necesitas un enfoque iterativo y escalable.
- Usa Lasso si quieres seleccionar características relevantes en datos pequeños o medianos.

- Usa Ridge si todas las características son importantes y el conjunto de datos no es demasiado grande.

Regresión regularizada: método directo vs. método estimado

Cómo se resuelven mínimos cuadrados, Ridge y Lasso según el enfoque de optimización

	Método directo (álgebra matricial)	Método estimado (descenso del gradiente)
Sin regularización	`LinearRegression()`	`SGDRegressor(penalty=None)`
Ridge (L2)	`Ridge(alpha=...)`	`SGDRegressor(penalty='l2')`
Lasso (L1)	`Lasso(alpha=...)`	`SGDRegressor(penalty='l1')`

Ridge

Tiene solución cerrada: $\beta = (X^TX + \lambda I)^{-1}X^Ty$. Encaja de forma natural en ambos métodos.

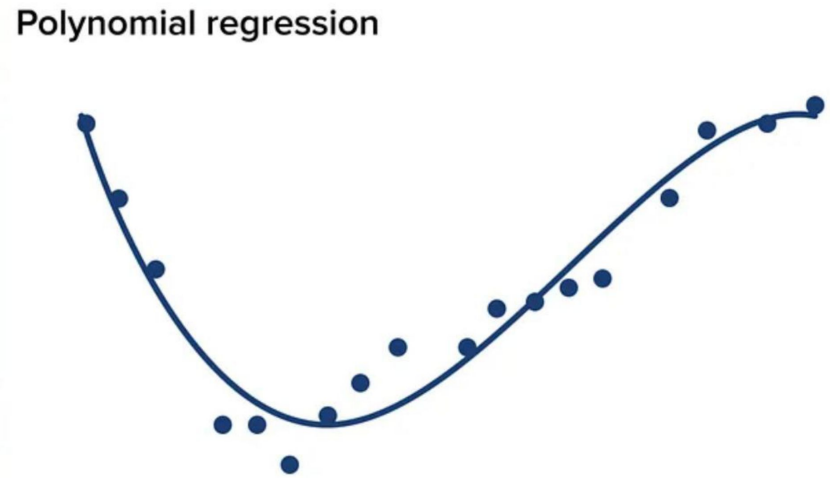
Lasso

El término $|\beta|$ no es derivable en cero: no existe fórmula matricial. Requiere coordinate descent u otros algoritmos, no gradiente estándar.

Regresión polinómica

En ocasiones los datos se puede ajustar mejor a una relación no-lineal. En este caso se puede utilizar la regresión polinómica.
La regresión polinómica, al igual que la lineal, trata de predecir el valor de la variable objetivo a partir de estimar unos pesos para cada una de las variables input más un término independiente (intercept)

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_1 x_2 + \theta_5 x_2^2 + \ldots + \theta_n x_m^p$$



Al igual que en la regresión lineal el objetivo es minimizar el MSE

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Regresión polinómica

En python scikit learn existe Polynomial Features que lo que hace es crear features nuevas a partir de las originales y elevadas al cuadrado, cubo, etc.

Y sobre este nuevo set de datos se aplica una regresión lineal.

```
from sklearn.preprocessing import PolynomialFeatures

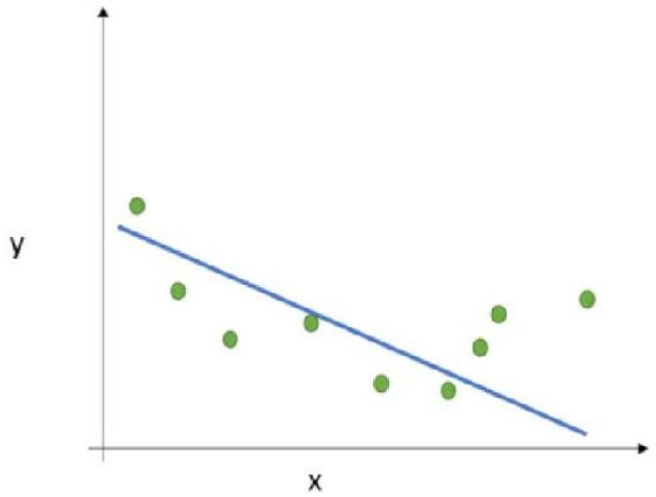
poly_features = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly_features.fit_transform(X)

model = LinearRegression()
model.fit(X_poly , y)
```

PolynomialFeatures añade todas las combinaciones posibles. Por ejemplo si definimos grado 3(degree=3) añade: a2, b2, a3, b3, ab, ab2, a2b, ab, ...

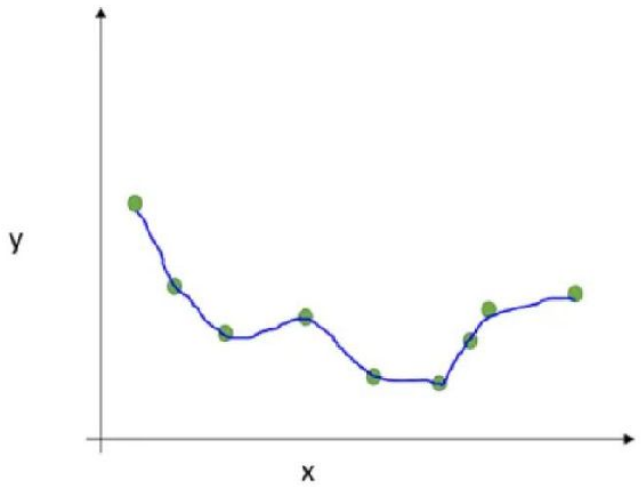
Overfitting

A
underfit



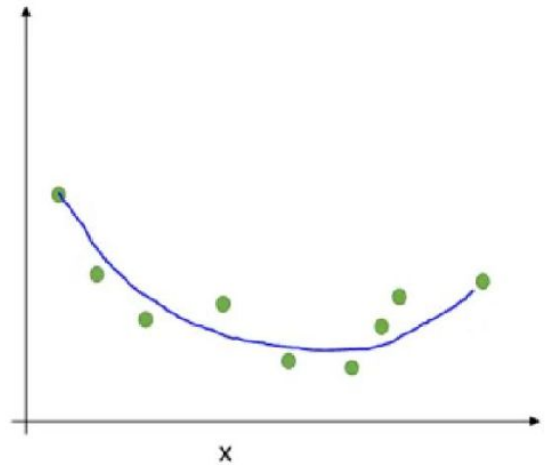
$$y_{pred} = \theta_0 + \theta_1 * x$$

B
overfit



$$y_{pred} = \theta_0 + \theta_1 * x + \theta_2 * x^2 + \theta_3 x^3 + \theta_4 x^4$$

C
balanced



$$y_{pred} = \theta_0 + \theta_1 * x + \theta_2 * x^2$$

Test and Train

Cuando estamos desarrollando un modelo nos interesa separar la muestra en tres y entrenamiento, de tal modo que los datos de entrenamiento los emplearemos para construir el modelo y los de test para evaluar el modelo sabiendo que conocemos el valor de la variable objetivo.

Esto permite identificar el overfitting.

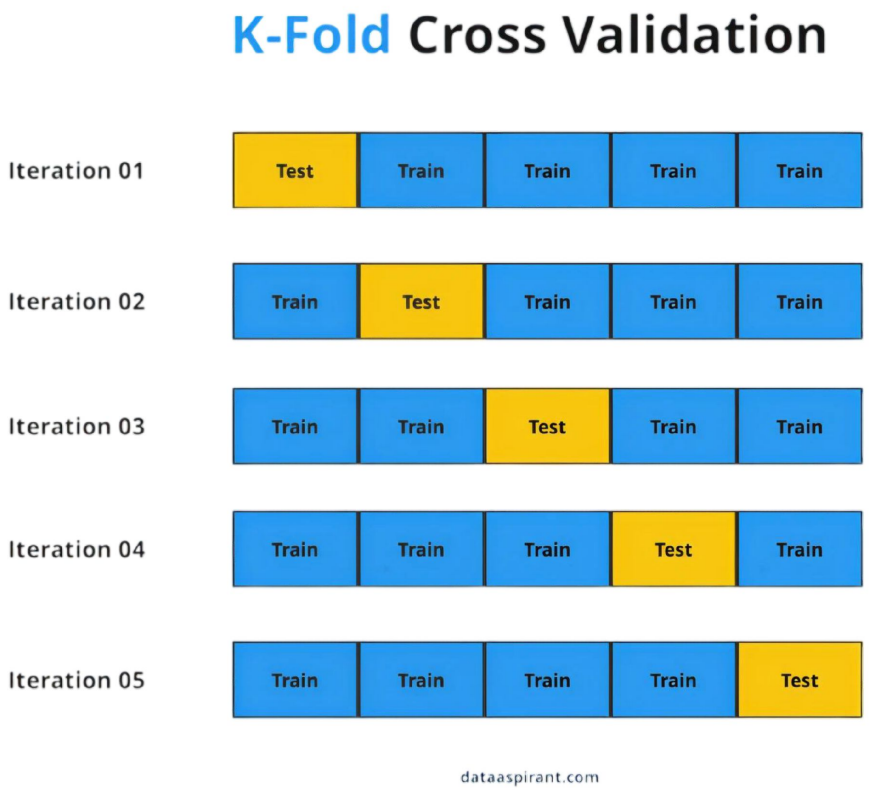
```
from sklearn.model_selection import train_test_split
xTrain, xTest, yTrain, yTest = train_test_split(x, y, test_size = 0.2, random_state = 0)
```

- Test_size: es el porcentaje de muestra que dejamos para test
- Random_state: es la semilla para la selección aleatoria de la muestra de test. Si usamos siempre la misma semilla aplicaremos siempre la misma división de los datos

Validación Cruzada (Cross Validation)

Mediante la validación cruzada podemos evaluar de una manera más robusta el modelo evitando depender de una única partición de datos.

1. Dividir el dataset en K subconjuntos (folds).
2. Entrenar el modelo K veces, cada vez usando un fold distinto como test y el resto como train.
3. Calcular la media de las métricas obtenidas.



Cross Validation

```
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier

# Validación cruzada con AUC
scores = cross_val_score(model, X, y, cv=5, scoring='roc_auc')
print("AUC promedio:", scores.mean())
```

Cross Validation estratificado

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=cv, scoring='roc_auc')
```

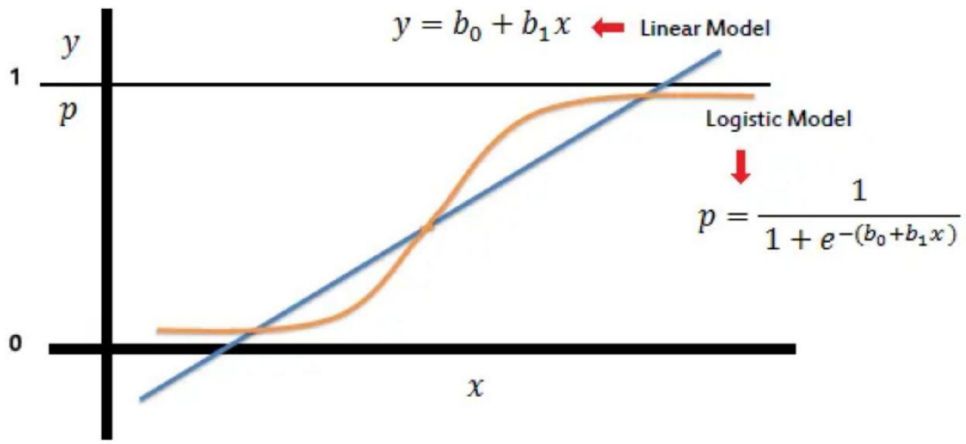
Regresion logistica

Básicamente es una regresión lineal con una función de activación al final que nos sirve para predecir una clase u otra.

Sirve para clasificar: Ej: el cliente contrata el préstamos o no.

```
from sklearn.model_selection import train_test_split
xTrain, xTest, yTrain, yTest = train_test_split(x, y, test_size = 0.2, random_state = 0)
```

- **Regresión Logística Binaria:** es la Regresión Logística clásica, en la que hay dos clases a predecir.
- **Regresión Logística Multinomial:** hay más de dos categorías a predecir, pero las clases no guardan ningún orden entre ellas.
- **Regresión Logística Ordinal:** hay más de dos categorías a predecir y existe un orden entre las categorías (por ejemplo predecir en qué posición va a quedar cada equipo al final de la liga)



Regression Logistica en Python: parámetros

penalty (default='l2'): Define la penalización para la regularización.Opciones: 'l1', 'l2', 'elasticnet', 'none'.Ejemplo: 'l1' para Lasso o 'elasticnet' para una combinación de Lasso y Ridge.

dual (default=False): Si se usa la formulación dual del problema. Solo aplicable cuando penalty='l2' y el solver es 'liblinear'. Opciones: True, False.

tol (default=1e-4): Define la tolerancia para el criterio de convergencia.

C (default=1.0): Parámetro inverso de la regularización (1/λ). Un valor más alto implica menos regularización. Ejemplo: C=0.1 refuerza la regularización.

fit_intercept (default=True): Especifica si se debe ajustar el término independiente (intercepto).

intercept_scaling (default=1) Escala aplicada al término intercepto, aplicable solo cuando el solver es 'liblinear'.

class_weight (default=None) Pondera las clases para manejar desequilibrio.Se emplea cuando la variable objetivo está muy desbalanceada, esto es que hay muy pocos casos Opciones: 'balanced' o un diccionario {class_label: **weight}.**

Regression Logistica en Python: parámetros

random_state (default=None) Semilla para la generación de números aleatorios, útil para reproducibilidad.

solver (default='lbfgs') Método de optimización para ajustar los coeficientes. Opciones: 'newton-cg', 'lbfgs', 'liblinear', 'sag', 'saga'. Importante: El tipo de penalización afecta qué solvers están disponibles.

max_iter (default=100) Número máximo de iteraciones del solver.

multi_class (default='auto') Define cómo manejar problemas multiclase. Opciones: 'auto', 'ovr' (uno contra el resto), 'multinomial'.

n_jobs (default=None) Número de CPUs usadas para el cálculo paralelo (aplicable para algunos solvers).

l1_ratio (default=None) Define la mezcla de L1 y L2 para regularización cuando se usa 'elasticnet'. Debe estar entre 0 y 1.

```
from sklearn.linear_model import LogisticRegression
logmodel = LogisticRegression(max_iter=1000)
logmodel.fit(X_train,y_train)
predictions = logmodel.predict(X_test)
```

Evaluación de modelos: métricas

Confusion matrix

		Actual values: Yes	Actual values: No
Predicted values	**Yes**	True Positive (TP)	False Positive (FP)
	No	False Negative (FN)	True Negative(TN)

La confusion matrix nos ayuda a ver de un simple vistazo como de bien nuestro modelo estima cada una de las clases.

```
# Crear la matriz de confusión
cm = confusion_matrix(y_true, y_pred)

# Mostrar la matriz de confusión
print("Matriz de Confusión:")
print(cm)
```

Classification Report

from sklearn.metrics import
classification_report
print(classification_report(y_test,predictions))

Precisión: Una precisión alta indica que el modelo tiene pocos falsos positivos, es decir, no clasifica erróneamente casos negativos como positivos.

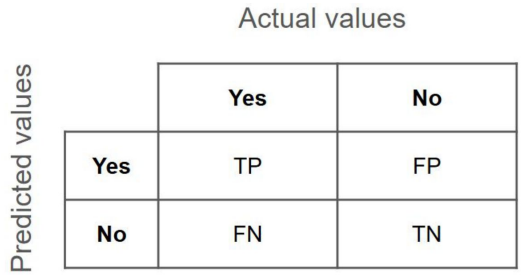
$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall (Sensibilidad o Exhaustividad): Un recall alto significa que el modelo identifica correctamente la mayoría de los casos positivos, minimizando los falsos negativos.

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score: Combina precisión y recall en una sola métrica, siendo más representativa cuando ambas son importantes.

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$



Support: es el número de ocurrencias reales para cada caso

Macro Average: Promedia las métricas (precisión, recall y F1) calculándolas para cada clase por igual, sin considerar el soporte de cada clase. Es útil cuando quieres evaluar el desempeño del modelo considerando todas las clases de la misma importancia.

Accuracy: Puede ser engañosa en casos de clases desbalanceadas porque solo mide el promedio global de aciertos.

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

Weighted Average: Promedia las métricas considerando el soporte de cada clase. Es decir, las clases con más ejemplos tienen más peso en el cálculo. Refleja mejor el desempeño del modelo en conjuntos de datos desbalanceados.

Otras métricas

ROC-AUC (Receiver Operating Characteristic - Area Under the Curve)

Definición: Mide la capacidad del modelo para distinguir entre clases. Representa el área bajo la curva ROC (True Positive Rate TPR vs False Positive Rate FPR).
Útil cuando: Quieres evaluar el desempeño general del modelo, especialmente en problemas con clases desbalanceadas.
Valores: 0.5 significa un modelo aleatorio; 1.0 es un modelo perfecto.

El índice Kappa (o Kappa de Cohen) mide el grado de acuerdo entre las predicciones de un modelo y los valores reales, ajustando por el acuerdo que podría ocurrir por azar.
El valor del índice Kappa oscila entre: 1: Acuerdo perfecto entre las predicciones y las etiquetas reales. 0: Acuerdo puramente aleatorio (sin relación real entre predicciones y etiquetas). Valores negativos: Indican un desacuerdo peor que el azar.
Este índice es útil cuando las clases están desbalanceadas o quieres evaluar la consistencia del modelo más allá de la precisión global.

Matthews Correlation Coefficient (MCC)

Definición: Mide la calidad de la clasificación considerando todas las categorías de la matriz de confusión.
Útil cuando: Quieres una métrica equilibrada incluso para datos desbalanceados.
Rango: De -1 (desempeño malo) a 1 (desempeño perfecto).

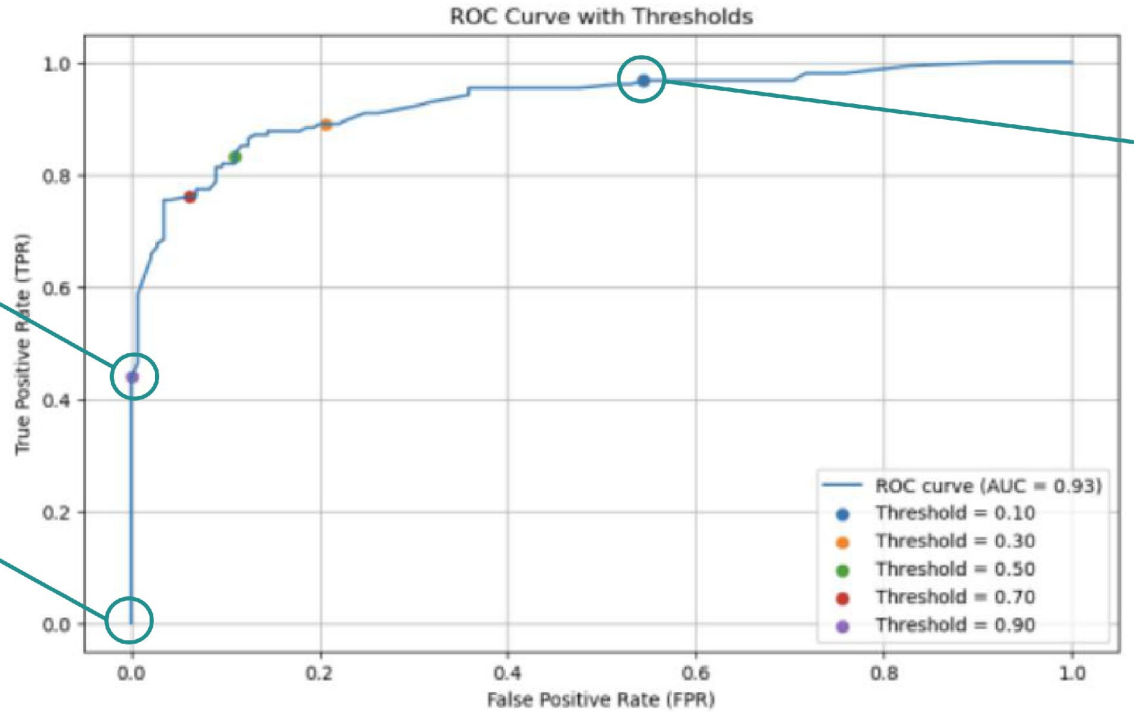
Curva Lift

Tiene un enfoque marketiniano. Viene a decirnos a cuantos positivos capturamos si nos dirigimos al X% con mayor probabilidad de contratar.

Curva ROC

Para un umbral de 0.9 prácticamente todo lo que capturamos es True positive y apenas hay falsos positivos.

Umbral 1



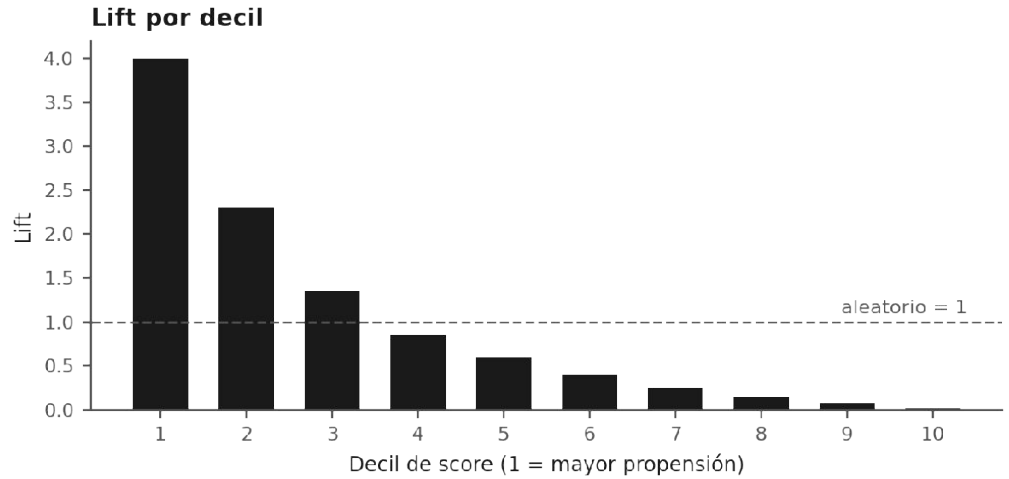
Para un umbral de 0.1 capturamos la casi totalidad de los positivos, sin embargo se nos llena de falsos positivos

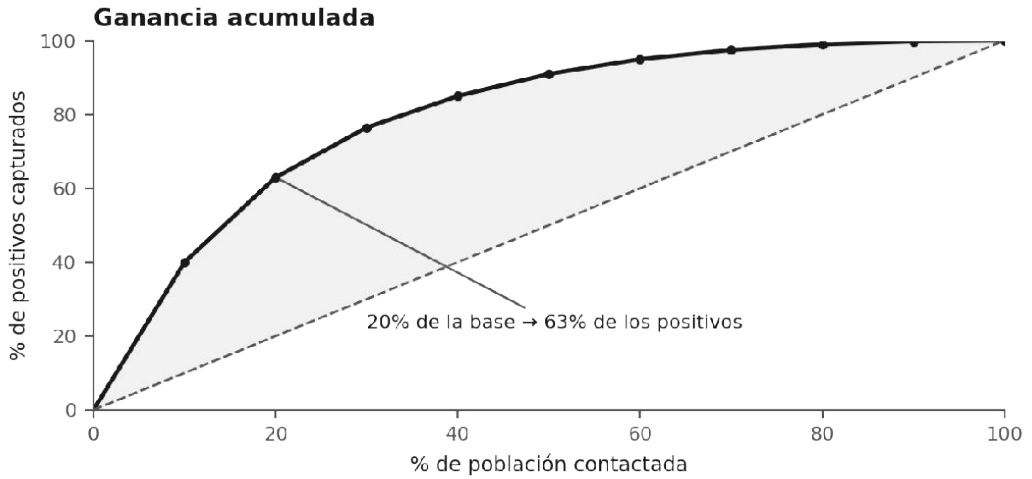
¿Qué muestra este gráfico?
La curva se va moviendo para cada umbral diferente empieza por umbral 1 donde nadie es clasificado como positivo.
Cada punto marcado (por ejemplo, Threshold = 0.1, 0.3, 0.5, etc.) representa cómo cambia el rendimiento del modelo al ajustar el umbral de clasificación:

- Umbral bajo (0.1): El modelo clasifica casi todo como positivo → alta TPR, pero también alta FPR.
- Umbral medio (0.5): Balance típico entre sensibilidad y especificidad.
- Umbral alto (0.9): El modelo solo clasifica como positivo si está muy seguro → baja FPR, pero también baja TPR.

Curva Lift y ganancia acumulada

Se ordena la población por score descendente, se divide en deciles y se compara la tasa de positivos de cada tramo con la tasa media. **Lift = 4 en el primer decil** significa que en el 10% con mayor score hay cuatro veces más positivos que en una selección aleatoria del mismo tamaño.





Por qué gusta a negocio
Responde a la pregunta que se hace de verdad: si solo puedo contactar al 20% de la base, ¿cuántos positivos capturo? Sirve para dimensionar campañas y estimar retorno sin hablar de probabilidades.

Por qué se ve menos ahora
Las decisiones han pasado de listas de campaña a decisiones cliente a cliente (Next Best Action, pricing). Ahí manda la probabilidad calibrada y el valor esperado, no el decil. Además scikit-learn no la trae de serie.

Muestras desbalanceadas

En banca el evento que queremos predecir suele ser raro: contratación, impago, fraude, baja.
Con un 2% de positivos, un modelo que prediga siempre "no" acierta el 98% de las veces y no sirve para nada.

Qué se rompe

- El **accuracy** deja de informar: mide la clase mayoritaria.
- El modelo optimiza la clase frecuente y aprende poco del evento que interesa.
- Con pocos positivos absolutos, las métricas de validación tienen mucha varianza.

Soluciones, en orden de coste

1. **No tocar los datos.** Elegir métricas adecuadas (AUC, PR-AUC, recall al X% de población contactada) y ajustar el umbral según el coste de un falso positivo frente a un falso negativo. Suele bastar.
2. **Ponderar las clases.** `class_weight='balanced'` en LogisticRegression, RandomForest o SVM; `scale_pos_weight = n_neg / n_pos` en XGBoost. No inventa ni descarta datos.
3. **Remuestrear.** Submuestreo de la mayoritaria (rápido, pierde información) o sobremuestreo sintético tipo SMOTE (riesgo de ruido en zonas de frontera). Solo dentro del train de cada fold, nunca en test.
4. **Recalibrar.** Si has remuestreado, las probabilidades quedan desplazadas: sirven para ordenar, no como probabilidad real. Platt o isotónica para devolverlas a su escala. En resumen, es devolver las probabilidades a su escala inicial tras haber sido remuestreada.

PR-AUC: ¿qué tan bien detecta el modelo los positivos?

Precision vs. Recall en cada umbral de decisión — útil cuando la clase positiva es minoritaria. Se va variando el threshold y calcula la precision y el recall para cada threshold

Precision

De todo lo que el modelo marcó como positivo, ¿cuánto acertó? Evita falsas alarmas.

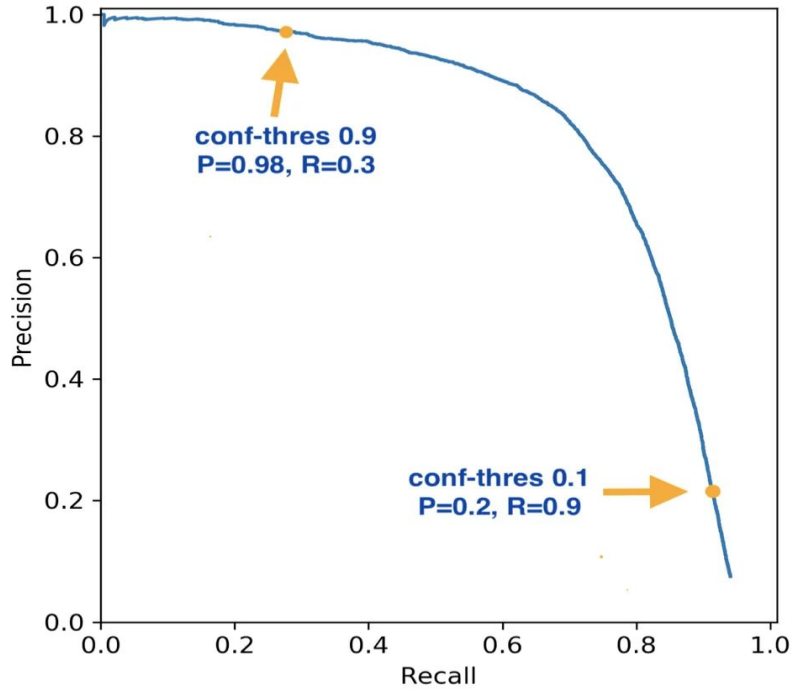
Recall

De todos los positivos reales, ¿cuántos detectó el modelo? Evita dejar casos sin detectar.

Punto de referencia

- PR-AUC del modelo vs. proporción de positivos (baseline) → si tu PR-AUC = 0.35 y tus positivos son el 1% → tu modelo es muy bueno (35 veces mejor que el azar).
- Si tu PR-AUC = 0.35 pero tus positivos son el 30% → tu modelo apenas mejora el azar.

Curva Precision–Recall

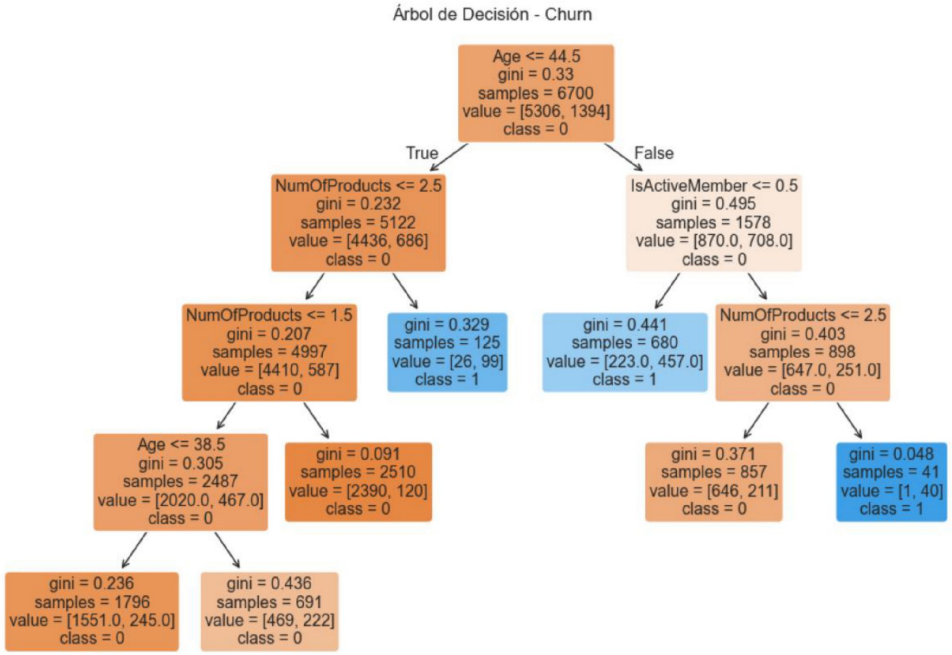


El área bajo esta curva (PR-AUC) resume el trade-off en un solo número: más cerca de 1 = mejor equilibrio entre precision y recall en todos los umbrales.

Árboles de decisión

Los **árboles de decisión** se utiliza tanto para problemas de clasificación como de regresión. Funcionan dividiendo los datos en subconjuntos más pequeños organizados en forma de un árbol, donde cada nodo representa una decisión basada en una característica, cada rama corresponde a un resultado de esa decisión, y cada hoja representa una etiqueta de clase (en clasificación) o un valor de predicción (en regresión).

Para cada nodo se selecciona la característica que mejor separa los datos según algún criterio (entropía, **indice de Gini,** reducción de suma de errores al cuadrado, ...). Este proceso se repite de manera recursiva hasta que los datos en cada grupo sean suficientemente homogéneos o se alcance algún límite predefinido, como la profundidad máxima del árbol.



- En Scikit Learn se aplica el algoritmo **CART** que tiene por objetivo reducir la impureza de los nodos.
- Permite trabajar con **variables categóricas y numéricas.**
- Uno de los parámetros que se define del árbol es el **número de ramas.**
- Tiene tendencia a **sobreajustar.**
- Puede ser **sensible a pequeñas variaciones de los datos.**
- Existen técnicas que combinan los árboles de decisión: **Random Forest o Gradient Boost.**
- Puede ser muy útil en **exploración de datos dada su explicabilidad**

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
dtree = DecisionTreeClassifier(max_depth=3, random_state=33)
dtree.fit(X_train,y_train)
```

Árboles de decisión en Python: Parámetros

`criterion` (default='gini')

- Método para medir la calidad de las divisiones.
  Opciones: `'gini'` (índice Gini), `'entropy'` (ganancia de información), y `'log_loss'` (para problemas de clasificación).

`splitter` (default='best')

- Estrategia para dividir en los nodos.
  Opciones: `'best'` (elige la mejor división) y `'random'` (elige división aleatoria).

`max_depth` (default=None)

- Profundidad máxima del árbol. Limitar la profundidad ayuda a prevenir el sobreajuste.

`min_samples_split` min_samples_split

- Número mínimo de muestras requeridas para dividir un nodo.

`min_samples_leaf` (default=1)

- Número mínimo de muestras que debe haber en un nodo hoja.

`min_weight_fraction_leaf` (default=0.0)

- Fracción mínima de peso requerida en un nodo hoja (útil para datos con pesos).

`max_features` max_features

- Número máximo de características consideradas para encontrar la mejor división.
  Opciones: `None` (todas las características), número entero, fracción o `'auto', 'sqrt', 'log2'.`

`max_leaf_nodes` (default=None)

- Número máximo de nodos hoja permitidos.

`min_impurity_decrease` (default=0.0)

- Valor mínimo de reducción de la impureza para realizar una división.

`random_state` (default=None)

- Semilla para la generación de números aleatorios, útil para reproducibilidad.

`class_weight` (default=None)

- Pondera las clases para manejar desequilibrio de clases.
  Opciones: `'balanced'` o un diccionario `{clase: peso}.`

`ccp_alpha` (default=0.0)

- Parámetro de poda para la reducción de costo-complejidad.
  Ayuda a simplificar el árbol eliminando nodos con beneficios insignificantes.

Arboles de decision: Índice de Gini

El índice de Gini es una métrica que se utiliza para medir la impureza o heterogeneidad de un conjunto de datos en árboles de decisión. En el contexto de machine learning, sirve para evaluar cómo se dividen los datos en un nodo con respecto a las clases objetivo.

Definición:

El índice de Gini se calcula como: $$G = 1 - \sum_{i=1}^{n} p_i^2$$

Donde:

- pi es la proporción (probabilidad) de los elementos que pertenecen a la clase ii en ese nodo.
- n es el número total de clases.

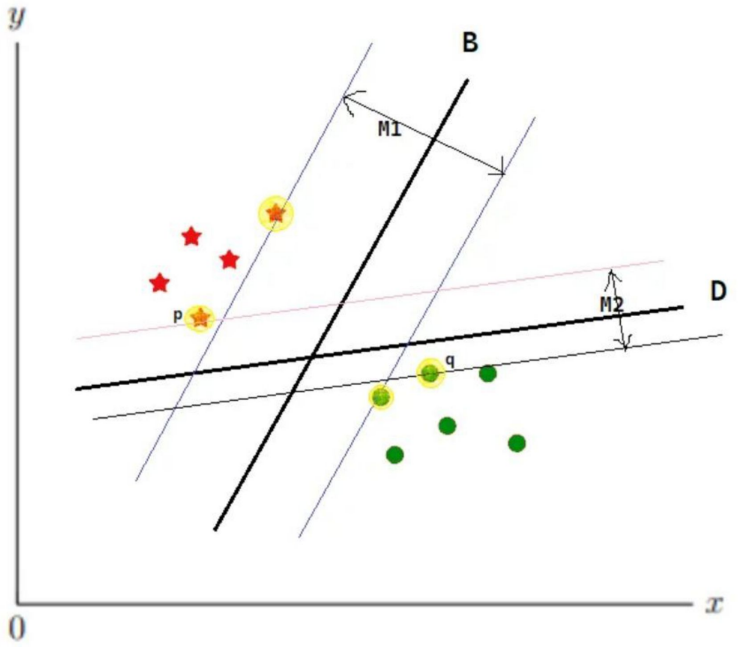
El índice de Gini varía entre:

- **0:** Representa la impureza mínima (el nodo contiene elementos de una sola clase, es completamente puro).
- **1:** Máxima impureza (las clases están distribuidas uniformemente en el nodo).

Support Vector Machine

La técnica de Support Vector Machine (SVM)se puede emplear tanto para clasificación como regresión. Su objetivo principal es encontrar un hiperplano óptimo que separe los datos en diferentes clases con el mayor margen posible:

- **Hiperplano óptimo:** SVM busca identificar un hiperplano que divida las distintas clases. En problemas bidimensionales, este hiperplano sería una línea, pero en dimensiones superiores se convierte en un espacio plano.
- **Máximo margen:** Llamamos margen es la distancia entre el hiperplano de separación y los puntos más cercanos de cada clase. SVM selecciona el hiperplano que maximiza el margen entre las clases, es decir, la distancia más cercana entre el hiperplano y los puntos de cada clase.
- **Vectores de soporte:** Sólo algunos puntos de datos, conocidos como vectores de soporte, influyen en la posición del hiperplano. Estos puntos son cruciales para definir el margen y la solución.
- **Kernels:** Cuando los datos no son linealmente separables en su espacio original, SVM utiliza funciones kernel, como el kernel radial o polinómico, para transformar los datos a un espacio dimensional más alto, donde sí pueden separarse linealmente.
- **Regularizacion:** Se le pueden añadir términos de regularización.



Es una técnica poderosa, especialmente cuando el conjunto de datos no es demasiado grande, y se destaca por su capacidad para trabajar bien con problemas complejos y datos de alta dimensionalidad.

`kernel`

- Especifica el tipo de kernel a usar. Los kernels son funciones que transforman los datos para hacerlos separables en un espacio diferente.
- Valores comunes: `'linear', 'poly', 'rbf', 'sigmoid'.`
- Por defecto: `'rbf'` (Radial Basis Function).

`C:`

- Controla la regularización. Un valor bajo de C permite un margen más amplio pero con más errores; un valor alto penaliza errores pero puede ajustarse demasiado a los datos.
- Valor por defecto: `1.0.`

`gamma`

- Define qué tan lejos puede influir un punto de entrenamiento. Valores altos significan que los puntos tienen influencia cercana, mientras que valores bajos tienen influencia más lejana.
- Solo aplica para kernels `'rbf', 'poly', o 'sigmoid'.`
- Valor por defecto: `'scale'.`

`degree`

- Es el grado del polinomio si el kernel es `poly'.`
- Valor por defecto: `3.`

`coef0`

- Es un parámetro independiente que afecta el kernel `poly' y 'sigmoid'.`
- Valor por defecto: `0.0.`

`probability`

- Si está activado `(True)`, permite la estimación de probabilidades.
- Por defecto: `False.`

`tol`

- Tolerancia para el criterio de detención durante el entrenamiento.
- Por defecto: `1e-3.`

`max_iter`

- Número máximo de iteraciones para la optimización. Si es `-1,` no hay límite.
- Valor por defecto: `-1.`

```
from sklearn.svm import SVC
svm_model = SVC(kernel='sigmoid')
svm_model.fit(X_train, y_train)
```

Naive Bayes

El clasificador Naive Bayes se basa en el teorema de Bayes, el cual es una fórmula matemática que describe la relación entre las probabilidades de los eventos y sus dependencias condicionales.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- P(A|B) es la probabilidad del evento A dado el evento B.
- P(B|A) es la probabilidad del evento B dado el evento A.
- P(A) es la probabilidad a priori del evento A.
- P(B) es la probabilidad a priori del evento B.

El clasificador Naive Bayes asume que las características de los datos son independientes entre sí, dado el etiquetado de clase: Naive assumption.

Naive Bayes funciona calculando la probabilidad de que una registro pertenezca a una clase particular basándose en las probabilidades de sus características individuales. Estima la probabilidad condicional de cada característica dada la clase y las multiplica para obtener la probabilidad de que la instancia pertenezca a esa clase. Al comparar las probabilidades para diferentes clases, el algoritmo asigna la instancia a la clase más probable.

Tipos de clasificadores Naive Bayes:

- **Gaussian Naive Bayes:** Asume que las características siguen una distribución normal (útil para datos numéricos).
- **Multinomial Naive Bayes:** Adecuado para datos discretos (como frecuencias de palabras en textos).
- **Bernoulli Naive Bayes:** Diseñado para datos binarios (como presencia o ausencia de características).

Se suele emplear más en casos donde tenemos textos o variables categóricas

Naive Bayes: ejemplo

Supongamos que tienes un conjunto de correos electrónicos que queremos clasificar si un correo es spam o no spam. Dado un correo con palabras específicas, el modelo determinará la probabilidad de que sea spam basándose en las ocurrencias de esas palabras en cada clase.

EJEMPLO

Probabilidad a priori:

P(spam)=0.3
P(no spam)=0.7

Probabilidad condicional de las palabras:

P(dinero|spam)=0.8
P(dinero|no spam)=0.1
P(rapido|spam)=0.6
P(rapido|no spam)=0.2

Suponemos que entra un correo nuevo con el asunto: "dinero rápido".

Calculamos las probabilidades para cada clase:

Spam:

$$P(spam|dinero,\ rápido) = P(dinero|spam) \cdot P(rápido|spam) \cdot P(spam) = 0.8 \cdot 0.6 \cdot 0.3 = 0.144$$

No Spam:

$$P(no\ spam|dinero,\ rápido) = P(dinero|no\ spam) \cdot P(rápido|no\ spam) \cdot P(no\ spam) = 0.1 \cdot 0.2 \cdot 0.7 = 0.014$$

Finalmente, se compara P(spam|dinero, rápido)P(spam|dinero,\ rápido) y P(no spam|dinero, rápido)P(no\ spam|dinero,\ rápido).
Dado que 0.144>0.014, el correo se clasificaría como **spam.Spam:**

Naive Bayes: parámetros

`priors`

Lista de probabilidades a priori para cada clase. Si no se especifica, las probabilidades se calculan automáticamente a partir de los datos.
python

`var_smoothing`

Un valor pequeño añadido a la varianza para evitar divisiones por cero. Valor por defecto:`1e-9.`
python

```
from sklearn.naive_bayes import GaussianNB
gnb_model =GaussianNB(var_smoothing=1e-9)
gnb_model.fit(X_train, y_train)
```

K Nearest Neighbours: KNN

El modelo se basa en observar cómo son los data points más próximos al caso que queremos estimar, y según como estén etiquetados estos data points próximos, esta será la categoría que asignaremos al nuevo

1

Paso 1

Selecciona el número K de vecinos. No hay una forma específica de determinar el mejor valor para "K", por lo que es necesario probar varios valores para encontrar el mejor. El valor más recomendado para K suele ser 5. Un valor muy bajo para K, como K=1 o K=2, puede ser ruidoso y los valores altos para K son buenos, pero pueden presentar ciertas dificultades de cálculo.

2

Paso 2

A continuación, calcula la distancia euclidiana entre los puntos de datos.

3

Paso 3

Toma los K vecinos más cercanos según la distancia euclidiana calculada.

4

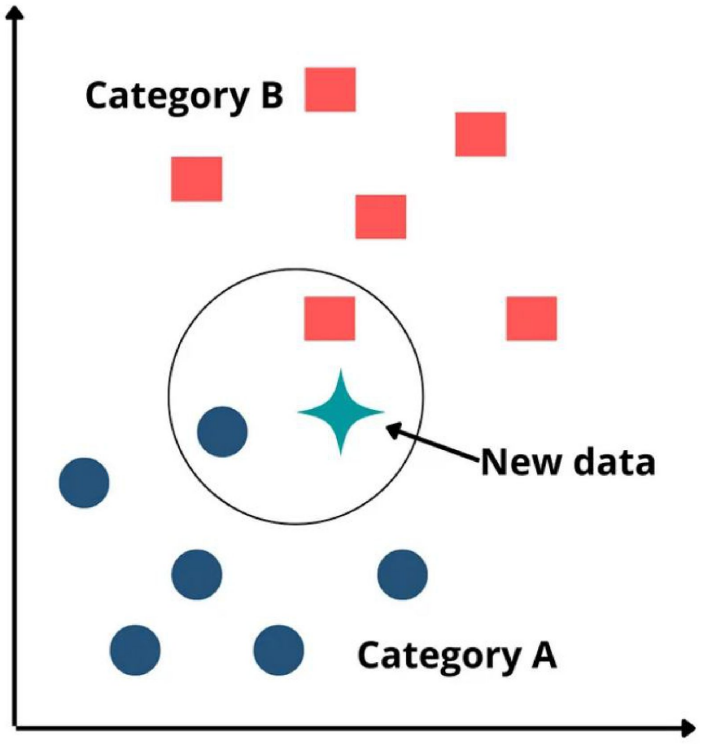
Paso 4

Entre estos K vecinos, cuenta el número de puntos de datos en cada categoría. K-NN asume que los puntos similares están más cerca entre sí.

5

Paso 5

Después de eso, asigna los nuevos puntos de datos a la categoría para la cual el número de vecinos es máximo. Agrupa los datos en la categoría más cercana al punto.



`n_neighbors`

El número de vecinos a considerar para clasificar cada punto.
Valor por defecto: **5.**

`weights`

- Determina cómo se ponderan los vecinos:
  - `'uniform'`: Todos los vecinos tienen el mismo peso.
  - `'distance'`: Los vecinos más cercanos tienen más peso.
  - Una función definida por el usuario también puede ser utilizada.

Valor por defecto: `'uniform'.`

`algorithm`

- El algoritmo utilizado para calcular los vecinos más cercanos:
  - `'auto'`: Escoge automáticamente el mejor algoritmo según los datos.
  - `'ball_tree'`: Utiliza estructuras tipo árbol (BallTree).
  - `'kd_tree'`: Utiliza estructuras tipo árbol (KDTree).
  - `'brute'`: Búsqueda de fuerza bruta.

Valor por defecto: `'auto'.`

`leaf_size`

- Tamaño de las hojas de los árboles (`BallTree o KDTree`). Afecta la velocidad de construcción y consulta.
- Valor por defecto: `30.`

`p`

- La métrica de distancia a usar:
  - p=1: Distancia de Manhattan.
  - p=2: Distancia Euclidiana (por defecto).
- Puedes ajustar pp para usar cualquier métrica basada en **Minkowski.**

`metric`

- Define la métrica de distancia:
  - Ejemplos: `'minkowski'` (por defecto), `'euclidean', 'manhattan',` etc.
  - También puede ser una función definida por el usuario.
- Valor por defecto: `'minkowski'.`

`metric_params`

- Parámetros adicionales para la métrica de distancia, si corresponde.
- Por defecto: **None.**

`n_jobs`

- Número de trabajos paralelos para realizar el cálculo.
  - `-1`: Usa todos los núcleos disponibles.
  - Valor por defecto: `None` (1 núcleo).

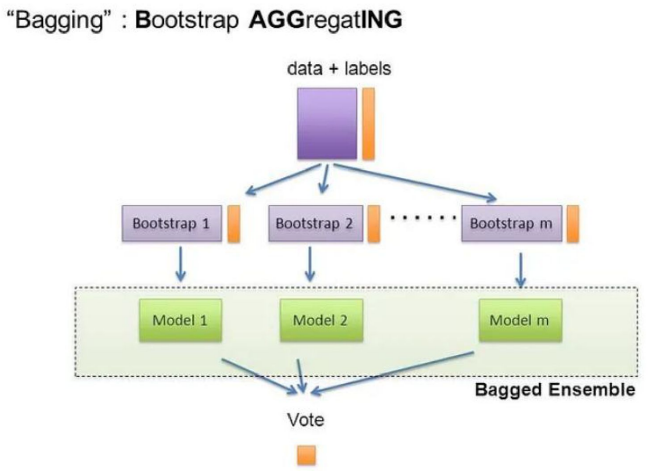
Ensemble models

Los ensemble models (modelos de conjunto) son técnicas de ML que combinan las predicciones de múltiples modelos base para mejorar el rendimiento general del modelo final.

En lugar de confiar en un solo modelo, la idea detrás de los ensembles es que al combinar varios modelos, se pueden mitigar sus debilidades individuales y aprovechar sus fortalezas, lo que generalmente resulta en mejores predicciones y mayor robustez.

¿Cómo funcionan los Ensemble Models?

Los modelos de conjunto operan bajo el principio de que una colección de modelos "débiles" (modelos que funcionan un poco mejor que el azar) puede ser combinada para producir un modelo "fuerte" (uno con alto rendimiento).



Modelos Ensemble

Principales tipos de Ensemble Models:

Bagging (Bootstrap Aggregating): Se generan múltiples modelos independientes entrenándolos en diferentes subconjuntos de los datos (con reemplazo).

Las predicciones de todos los modelos se combinan, generalmente con promedio (para regresión) o votación (para clasificación).
Ejemplo común: Random Forest.

Boosting: Construye modelos secuenciales, donde cada modelo intenta corregir los errores del modelo anterior. Los modelos finales tienen más peso en las observaciones difíciles de clasificar. Combina predictores débiles para obtener un predictor fuerte.

Importante: Algunos modelos de esta familia de modelos no está incluida en Scikit Learn (xGBoost, Catboost y LGBM por ejemplo)

Ejemplos de algoritmos: Gradient Boosting Machines (GBM), AdaBoost, XGBoost, LightGBM, CatBoost.

Stacking: Combina predicciones de diferentes modelos base mediante un modelo meta (como una regresión logística o SVM) que aprende a partir de las predicciones base. Las predicciones de los modelos base se usan como input en el modelo meta Es más flexible y puede usar diferentes tipos de modelos base.

Voting: Combina las predicciones de varios modelos diferentes (no necesariamente relacionados) con una votación: Votación mayoritaria (clasificación): Escoge la clase más votada. Votación promedio (regresión): Hace un promedio de las predicciones.

Random Forest

Random Forest es un modelo de aprendizaje automático por ensamble (Ensemble Learning) que combina, en este caso, múltiples árboles de decisión. Cada árbol en el bosque se entrena con una muestra aleatoria de los datos (muestreo bootstrap) y considera únicamente un subconjunto aleatorio de características al realizar divisiones (aleatorización de características).

Bootstrap sampling: Es una técnica de muestreo con reemplazo donde se toman muestras aleatorias de los datos originales, generalmente del mismo tamaño que a muestra original y donde puede haber repetidos.

En **clasificación,** el bosque predice mediante el voto mayoritario entre los árboles, mientras que en **regresión,** promedia las predicciones. La fortaleza del modelo radica en su enfoque de "sabiduría colectiva" **(wisdom** of **crowds).** El proceso de toma de decisiones colectivo tiende a compensar los errores que se pueden producir en árboles individuales y así generar predicciones más confiables.

```
from sklearn.ensemble import RandomForestClassifier
rfc = RandomForestClassifier(n_estimators=100)
rfc.fit(X_train, y_train)
```

Random Forest: parámetros

`n_estimators` (default=100)

- Número de árboles en el bosque.
- Aumentar este valor puede mejorar la precisión, pero también incrementa el tiempo de entrenamiento.

`criterion` (default='gini')

- Función para medir la calidad de las divisiones de cada árbol.
- Opciones: `'gini'` (índice Gini) o `'entropy'` (ganancia de información).

`max_depth` (default=None)

- Profundidad máxima de los árboles.
- Limitarlo puede prevenir el sobreajuste.

`min_samples_split` (default=2)

- Número mínimo de muestras requeridas para dividir un nodo.

`min_samples_leaf` (default=1)

- Número mínimo de muestras en un nodo hoja.

`min_weight_fraction_leaf` (default=0.0)

- Fracción mínima del peso total requerida en un nodo hoja (importante si las muestras tienen pesos).

`max_features` (default='sqrt')

- Número de características consideradas al buscar la mejor división.
- Opciones: `'auto'` (todas las características), `sqrt', 'log2'` o un valor entero/fracción.

`max_leaf_nodes` (default=None)

- Número máximo de nodos hoja permitidos.

`bootstrap` (default=True)

- Si es `True,` usa muestreo bootstrap (con reemplazo) para entrenar cada árbol.

`random_state` (default=None)

- Controla la aleatoriedad para la reproducibilidad.

`class_weight` **class_weight**

- Ponderación de las clases para problemas de clases desbalanceadas.
- Opciones: `'balanced'` o un diccionario`{clase: peso}.`

`n_jobs` (default=None)

- Número de procesadores usados en paralelo.
- `-1` usa todos los núcleos disponibles.

`oob_score` (default=False)

- Si es `True,` usa muestras fuera de bolsa (out-of-bag) para evaluar el modelo.

`ccp_alpha` ccp_alpha

- Parámetro de poda basado en la reducción de costo-complejidad.

`verbose` (default=0)

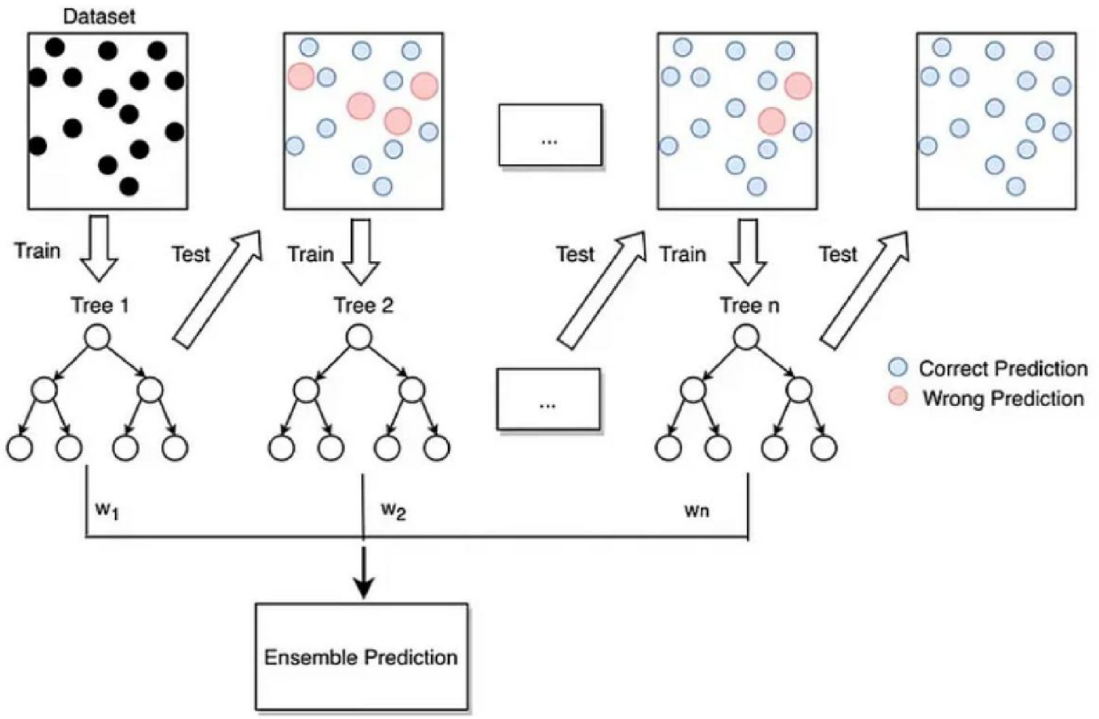
- Controla el nivel de información impresa durante el ajuste del modelo.

Gradient Boosting Models (GBM)

Como ya vimos, la técnica Boosting construye modelos secuenciales, donde cada modelo intenta corregir los errores del modelo anterior. Los modelos finales tienen más peso en las observaciones difíciles de clasificar:

1er árbol: F1(X, Y) → Y1
2º árbol: H1(X, Y-Y1) → Y2
3er árbol: H2(X, Y-Y2) → Y3

Las predicciones se actualizan sumando las contribuciones del nuevo árbol al modelo acumulado. Estas contribuciones se multiplican por un factor de aprendizaje **(wi),** que controla cuánto aporta cada árbol al modelo final.



eXtreme Gradient Boosting Machine (xGBoost)

Este es el más popular de los modelos GBM. Es una versión mejorada del GBM, más rápido y utilizando menos espacio, ya que no evalúa todas las posibles divisiones, sino solo algunas divisiones útiles.

Por ejemplo, si hay 1000 posibles puntos de división, elegirá solo los 100 mejores puntos, ahorrando espacio y tiempo (para ello emplea un algoritmo de división preordenado).

También se considera un GBM Regularizado, ya que se multiplica un término lambda (L) con la función del boosting (H1). Por lo tanto, la ecuación se convierte en L∗H1() en lugar de H1().

pip install xgboost

```
import xgboost as xgb
from xgboost import XGBClassifier
# Convertir los datos al formato DMatrix, que es el formato
optimizado de XGBoost


XGB_model = XGBClassifier()
XGB_model.fit(X_train, y_train)
```

eXtreme Gradient Boosting Machine: parámetros

1. **Parámetros principales (configuración del modelo)**

`n_estimators` Número de árboles (o iteraciones de boosting). Por defecto: 100.

`learning_rate` Tasa de aprendizaje para ajustar el impacto de cada árbol. Valores típicos: 0.01 a 0.3. Por defecto: 0.1.

`max_depth` Profundidad máxima de los árboles. Un valor más alto aumenta la complejidad. Por defecto: 6.

`booster` Especifica el tipo de booster a usar. Opciones:

- `'gbtree'` (por defecto): Árboles de decisión.
- `'dart'` Dropout en boosting.
- `'gblinear'` Modelos lineales.

2. **Parámetros para evitar sobreajuste**

`subsample` Proporción de muestras usadas para entrenar cada árbol. Valores típicos: 0.5 a 1. Por defecto: 1.

`colsample_bytree` Proporción de características seleccionadas para construir cada árbol. Valores típicos: 0.3 a 1. Por defecto: 1.

`gamma` Reducción mínima en la función de pérdida para dividir un nodo. Valores típicos: 0 a valores positivos (como 0.1). Por defecto: 0.

`min_child_weight` Peso mínimo de las instancias en un nodo hoja. Valores más altos reducen el sobreajuste. Por defecto: 1.

3. **Parámetros de regularización**

`reg_alpha` **(L1 regularization):** Controla la magnitud de los valores absolutos de los pesos. Por defecto: 0.

`reg_lambda` **(L2 regularization):** Controla la magnitud de los valores cuadrados de los pesos. Por defecto: 1.

eXtreme Gradient Boosting Machine: parámetros

4. **Parámetros específicos del objetivo**

`objective` Define la función objetivo. Ejemplos comunes:

- `'binary:logistic'` Para clasificación binaria.
- `'multi:softmax'` Para clasificación multiclase (devuelve una clase).
- `'multi:softprob'` Para clasificación multiclase (devuelve probabilidades para cada clase).

`eval_metric` Métrica de evaluación, como `'logloss'`, `'auc'` (área bajo la curva ROC), o `'error'`.

5. **Parámetros para manejo de datos grandes**

`tree_method` Algoritmo para construir árboles. Ejemplos:

- `'auto'` Selección automática (por defecto).
- `'hist'` Método basado en histogramas, más rápido para datos grandes.
- `'gpu_hist'` Usa GPU para acelerar el entrenamiento.

`max_bin` Número máximo de contenedores discretos para los histogramas (relevante para `tree_method='hist'` o `'gpu_hist'`).

6. Otros parámetros importantes

`use_label_encoder` Define si se usa el codificador de etiquetas interno de XGBoost. Desde XGBoost 1.3.0, se desactiva configurando `use_label_encoder=False`.

`seed` / `random_state` Fija la semilla para reproducibilidad.

Light Gradient Boosting Machine (LGBM)

LGBM también ha sido uno de los modelos emergentes que están ganando popularidad en el ámbito de la ciencia de datos.

Aunque la precisión de ambos modelos, XGB y LGBM, es bastante similar, su implementación es ligeramente diferente.

Para encontrar las mejores divisiones entre todas las posibles (aplicando el concepto de elegir 100 puntos de división de entre 1000, para reducir el trabajo extra), LGBM utiliza el muestreo unilateral basado en gradientes (GOSS), mientras que XGB usa un algoritmo preordenado con fines de división.

```
pip install lightgbm

import lightgbm as lgb
from lightgbm import LGBMClassifier

model = LGBMClassifier()

model.fit(X_train, y_train)

# Realizar predicciones
lgbm_pred = model.predict(X_test)
```

Light Gradient Boosting Machine: parámetros

1. **Parámetros principales (estructura del modelo)**

`n_estimators` Número de árboles (o iteraciones de boosting). Un valor más alto puede mejorar el rendimiento, pero podría aumentar el riesgo de sobreajuste. Valor por defecto: 100.

`learning_rate` Tasa de aprendizaje que controla cuánto contribuye cada árbol al modelo. Valores más bajos como 0.01 o 0.05 suelen ser buenos, aunque requieren más iteraciones. Valor por defecto: 0.1.

`max_depth` Profundidad máxima de los árboles. Un valor más alto puede capturar relaciones más complejas, pero también aumenta el riesgo de sobreajuste. Por defecto, se establece en -1 (sin límite).

`boosting_type` Método de boosting que se utilizará:

- `'gbdt'` (por defecto): Gradient Boosting Decision Trees.
- `'dart'` Dropout en boosting.
- `'rf'` Random Forest.

2. **Parámetros para prevenir sobreajuste**

`subsample` Proporción de datos seleccionados de manera aleatoria para entrenar cada árbol. Valores típicos: 0.6 a 1. Valor por defecto: 1 (usar todos los datos).

`colsample_bytree` Proporción de características seleccionadas para construir cada árbol. Un valor más bajo puede evitar el sobreajuste. Valor por defecto: 1.

`min_child_samples` Número mínimo de muestras en un nodo hoja. Un valor más alto reduce el riesgo de sobreajuste. Valor por defecto: 20.

`reg_alpha` **(L1 regularization):** Penaliza la complejidad del modelo añadiendo una regularización L1. Valores típicos: 0 a 1. Valor por defecto: 0.

`reg_lambda` **(L2 regularization):** Penaliza la magnitud de los pesos con regularización L2. Valores típicos: 0 a 1. Valor por defecto: 0.

Light Gradient Boosting Machine: parámetros

3. **Parámetros específicos del objetivo**

`objective` Define la función objetivo del modelo. Ejemplos comunes:

- `'binary'` Clasificación binaria.
- `'multiclass'` Clasificación multiclase.
- `'cross_entropy'` Para clasificación multiclase con probabilidades.

`metric` Métrica para evaluar el modelo. Ejemplos comunes:

- `'logloss'` (logaritmo de pérdida, para clasificación binaria).
- `'multi_logloss'` (para problemas multiclase).
- `'auc'` (área bajo la curva ROC).

4. **Parámetros adicionales (optimizaciones avanzadas)**

`num_leaves` Número máximo de hojas en los árboles. Controla la complejidad del modelo; más hojas permiten capturar interacciones más complejas.
Valores típicos: 31 o 64.

`max_bin` Número de contenedores discretos para dividir los valores continuos. Ajustar este valor puede mejorar el rendimiento.
Valor por defecto: 255.

`early_stopping_rounds:` Detiene el entrenamiento si la métrica no mejora después de un número específico de iteraciones.

Categorical Boost (Catboost)

CatBoost, no es tan popular como los anteriores.

Es comparativamente más lento que LGBM y XGB, pero tiene una ventaja inigualable: puede recibir datos categóricos en forma de texto (solo necesitas indicar cuáles columnas son categóricas) y entrenar el modelo, de ahí el nombre Categorical Boost.

La particularidad es que entiende los datos categóricos, mientras que otros modelos simplemente los toman como datos numéricos. No se requiere un paso de preprocesamiento para convertir el texto a numérico usando OneHotEncoder o LabelEncoding para las categorías, lo que le permite producir mejores resultados.

Abril 25 → catboost no es compatible con python 3.13

pip install catboost

```
from catboost import CatBoostClassifier


# Crear y entrenar el modelo CatBoost usando la API de
sklearn
model = CatBoostClassifier()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

Categorical Boost: Parámetros

1. Parámetros principales (configuración del modelo)

`n_estimators` Número de árboles (o iteraciones de boosting). Por defecto: 100.

`learning_rate` Tasa de aprendizaje para ajustar el impacto de cada árbol. Valores típicos: 0.01 a 0.3. Por defecto: 0.1.

`max_depth` Profundidad máxima de los árboles. Un valor más alto aumenta la complejidad. Por defecto: 6.

`booster` Especifica el tipo de booster a usar. Opciones:

- `'gbtree'` (por defecto): Árboles de decisión.
- `'dart'` Dropout en boosting.
- `Dropout en boosting.` Modelos lineales

2. Parámetros para evitar sobreajuste

`subsample:` Proporción de muestras usadas para entrenar cada árbol. Valores típicos: 0.5 a 1. Por defecto: 1.

`colsample_bytree` Proporción de características seleccionadas para construir cada árbol. Valores típicos: 0.3 a 1. Por defecto: 1.

`gamma` Reducción mínima en la función de pérdida para dividir un nodo. Valores típicos: 0 a valores positivos (como 0.1, 1).

`min_child_weight` Peso mínimo de las instancias en un nodo hoja. Valores más altos reducen el sobreajuste. Por defecto: 1.

3. Parámetros de regularización

`reg_alpha` **(L1 regularization):** Controla la magnitud de los valores absolutos de los pesos. Por defecto: 0.

`reg_lambda` **(L2 regularization)** Controla la magnitud de los valores cuadrados de los pesos. Por defecto: 1.

Categorical Boost: Parámetros

4. **Parámetros específicos del objetivo**

`objective` Define la función objetivo. Ejemplos comunes:

- `'binary:logistic'` Para clasificación binaria.
- `'multi:softmax'` Para clasificación multiclase (devuelve una clase).
- `'multi:softprob'` Para clasificación multiclase (devuelve probabilidades para cada clase).

`eval_metric` Métrica de evaluación, como `'logloss'`, `'auc'` (área bajo la curva ROC), o `'error'`.

5. **Parámetros para manejo de datos grandes**

`tree_method` Algoritmo para construir árboles. Ejemplos:

- `'auto'` Selección automática (por defecto).
- `'hist'` Método basado en histogramas, más rápido para datos grandes.
- `'gpu_hist'` Usa GPU para acelerar el entrenamiento.

`max_bin` Número máximo de contenedores discretos para los histogramas (relevante para `tree_method='hist'` o `'gpu_hist'`).

6. **Otros parámetros importantes**

`use_label_encoder` Define si se usa el codificador de etiquetas interno de XGBoost. Desde XGBoost 1.3.0, se desactiva configurando `use_label_encoder=False`.

`seed / random_state` Fija la semilla para reproducibilidad.